

Maritime CargoService: Sprint 1

Davide Chirichella, Gabriele Doti, Daniele
Maccagnan

Ingegneria dei Sistemi Software, A.A. 2025/2026
Alma Mater Studiorum . Università di Bologna

June 26, 2026

1. Introduction

Il punto di partenza di questo sprint è l'architettura logica e l'analisi dei requisiti definita nello Sprint 0 (recuperabile al seguente [link](#)^o). Si riporta di seguito il goal dello sprint 1:

Realizzare un primo prototipo eseguibile del **cargoservice** che implementi il comportamento principale descritto dai requisiti mediante collaboratori simulati.

Al termine dello Sprint1 il sistema dovrà essere in grado di:

- ricevere una **load_request** proveniente dall'IOPort;
- verificare lo stato della hold, dell'IOPort e del servizio;
- produrre una delle tre risposte previste:
 - **load_accepted(slotID)**;
 - **load_retrylater**;
 - **load_refused**;
- mantenere il modello logico della hold e la prenotazione dello slot;
- gestire gli stati **engaged** e **disengaged**;
- pilotare il LED in funzione dello stato del sistema;
- superare i test funzionali definiti nello Sprint0.

In questa fase il cargorobot, il sonar, il markerdevice e l'IOPort saranno rappresentati tramite collaboratori simulati.

2. Requirements

L'azienda richiede di realizzare un servizio denominato **cargoservice** con il seguente funzionamento:

- Il cargoservice è in grado di ricevere una richiesta di carico di un container inviata da un cliente tramite il pulsante dell'IOPort.
- Invia la risposta **retrylater** se l'IOPort è attualmente occupato da un container oppure se il sistema è **Out of service**.
- Rifiuta la richiesta quando la hold è già piena, ovvero gli slot1-4 sono già tutti occupati.
- Altrimenti, considera il sistema come **engaged**, rileva uno slot libero e restituisce come risposta il nome dello slot riservato. Mentre il sistema è engaged, il LED deve lampeggiare.
- Quando la richiesta di carico viene accettata, il cliente deve spostare il container nell'area del sensore entro un tempo prefissato (ad esempio 30 secondi), altrimenti il sistema diventa **disengaged**.
- Successivamente, il cargoservice utilizza il cargorobot per spostare il container dall'IOPort a slot5 (per l'etichettatura del container) e poi allo slot riservato.
- Il servizio deve inoltre mostrare sul display dell'IOPort:
 - lo stato attuale della hold
 - il messaggio "**Service working**" quando tutto sta procedendo correttamente
 - il messaggio "**Out of service**" se il sensore sonar misura una distanza $D > D_{FREE}$ per almeno 3 secondi (possibile guasto del sonar)

3. Requirement analysis

L'analisi dettagliata del dominio e dei requisiti è già stata affrontata nello Sprint 0, in questa fase ci concentriamo esclusivamente sul ciclo di gestione delle richieste di carico (**core business**) del sistema.

Come emerso in precedenza, il fulcro del sistema è l'attore **cargoservice**, la cui responsabilità principale è fungere da orchestratore per coordinare le operazioni.

Tuttavia, i requisiti affrontati in questo sprint presupporrebbero già l'implementazione e la presenza di altri componenti del sistema, come la stiva (hold), i sensori (sonar), i dispositivi di I/O (IOPort, LED, markerdevice) e l'interazione con l'hardware del robot (cargorobot). Per focalizzarci unicamente sulla logica di business e rispettare un processo di costruzione incrementale, in questo Sprint 1 verranno utilizzati dei componenti **simulati**

L'uso del linguaggio qak ci permetterà di modellare il **cargoservice** come un **attore autonomo**.

4. Problem analysis

Come emerso dai requisiti, questo componente funge da **orchestratore**: coordina le operazioni degli altri componenti del sistema al fine di eseguire le procedure di carico nel giusto ordine

5. Test plans

6. Project

7. Testing

8. Deployment

9. Maintenance

10. Team di lavoro

NOME E COGNOME	MATRICOLA
Davide Chirichella	0001222371
Gabriele Doti	0001245897
Daniele Maccagnan	0001247340